

서울과학기술대학교
SEOULTECH Seoul National University of Science & Technology

AI와 딥러닝을 활용한 경영분석

Week 7

Topics

- 로지스틱 회귀분석 및 모델 평가
- 리샘플링
- 하이퍼 파라미터 튜닝

데이터 타입에 따른 분석 모델

독립변수 X	종속변수 Y	분석방법
계량/범주형	계량형	회귀분석
계량/범주형	범주형	로지스틱 회귀분석이나 머신러닝 알고리즘들 (의사결정트리, RF, LDA, kNN, SVM, NN등)
범주형	계량형	t-test (범주가 2개) ANOVA (범주가 3개 이상)
범주형	범주형	카이스퀘어 테스트 머신러닝 알고리즘 (베이지안)

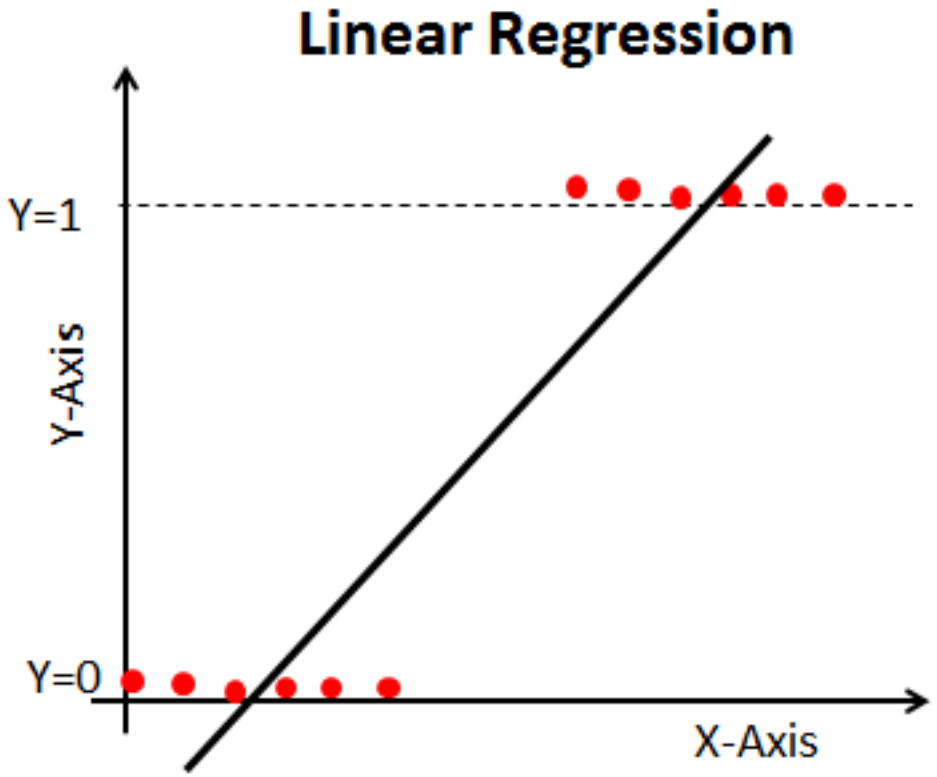
Logistic Regression
로지스틱 회귀분석

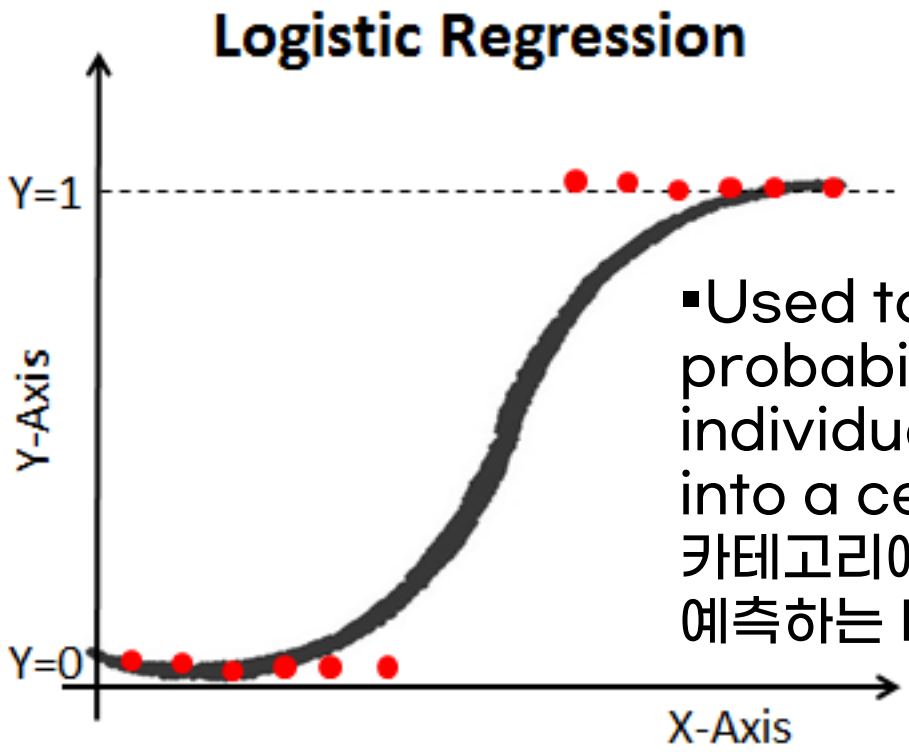
- A supervised learning classification algorithm used to predict the probability of a target variable. 종속변수의 확률을 예측하는 데 사용되는 분류 알고리즘
- The nature of target or dependent variable is dichotomous, which means there would be only two possible classes. 2개의 클래스를 가진 문제에 적합

Binary or Binomial	Multinomial	Ordinal
• success or failure, yes or no, win or loss	• “type A” or “type B” or “type C”	• “poor” or “good”, “very good”, “excellent”

$$\frac{1}{1+e^{-x}}$$

Logistic Function 로지스틱 함수





- Used to predict the probability that an individual observation falls into a certain category. 어떤 카테고리에 속하는 지 확률을 예측하는 데 사용됨

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p$$

$$p(X) = e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p} / (1 + e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p})$$

Logistic Regression 로지스틱 회귀

```
from sklearn.linear_model import
LogisticRegression

log_model = LogisticRegression()
log_model.fit(x_train, y_train)
log_model.coef_ #기울기
log_model.intercept_ #y절편
```

Classication Model Evaluation
분류모델 평가

```
log_model.score(x_test, y_test) #정확도

from sklearn.metrics import accuracy_score
y_pred = log_model.predict(x_test)
log_score = accuracy_score(y_test, y_pred)
```

Accuracy 정확도

- The percentage of correctly classifies instances out of all instances. 전체 데이터 중에서 올바르게 분류된 데이터의 퍼센트
- It is more useful on a binary classification than multi-class classification problems. 다중 클래스 분류 문제보다 이진 분류 문제에 더 유용

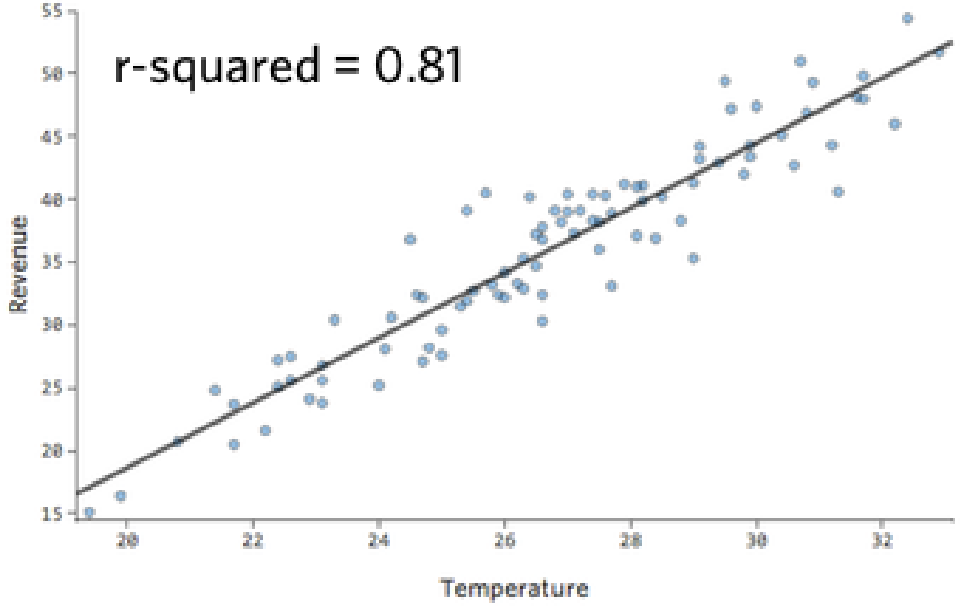
Linear Regression 선형 회귀

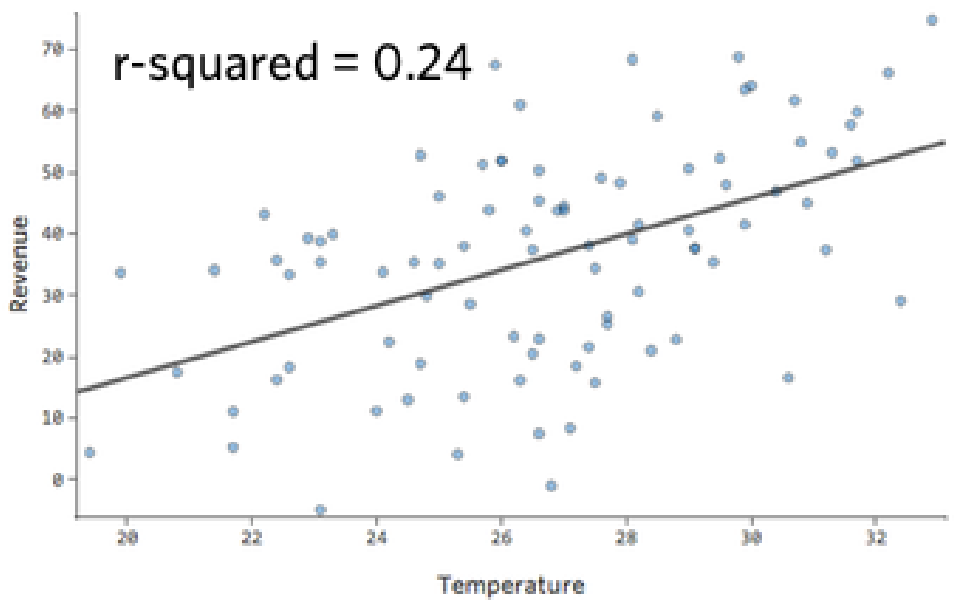
```
from sklearn.linear_model import
LinearRegression


lin_model = LinearRegression()
lin _model.fit(x_train, y_train)
lin _model.coef_ #기울기
lin _model.intercept_ #y절편
```

R^2 결정계수

- R Squared, also called the coefficient of determination. 알스퀘어드 또는 결정계수라고 불림
- Provides a “goodness of fit” measure for the predictions to the observations. 관측치와 예측이 잘 맞는지 보여줌





Regression Model Evaluation
회귀모델 평가

```
lin _model.score(x_test, y_test) #정확도

from sklearn.metrics import r2_score
y_pred = lin _model.predict(x_test)
log_score = r2_score(y_test, y_pred)
```

Confusion Matrix 혼동행렬

Contingency Table이라고도 함

Predicted 예측값

	negative	positive
negative	TN	FP
positive	FN	TP

Actual
실제값

Confusion Matrix Example
혼동행렬 예시

- 201 1's correctly classified as "1"
- 85 1's incorrectly classified as "0"
- 25 0's incorrectly classified as "1"
- 2689 0's correctly classified as "0"

Classification Confusion Matrix		
	Predicted Class	
Actual Class	1	0
1	201	85
0	25	2689

Code For Confusion Matrix 코드

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test,y_pred)
```

```
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]
```

Error Rate 오차율

- Error Rate = (sum of misclassified records)/(total records) 오차율 = (잘못 분류된 레코드의 숫자의 합)/(전체 레코드의 수)
- Overall error rate = (25+85)/3000 = 3.67%

Classification Confusion Matrix		
	Predicted Class	
Actual Class	1	0
1	201	85
0	25	2689

오차들

Accuracy 정확도

- Accuracy 정확도
  전체에서 맞는게 얼마나 되는지 봄
  = (TP + TN) / (TP + TN + FP + FN)
  = (201+2689)/3000
  = 1 - error = 1 - 3.67% = 96.33%

Classification Confusion Matrix		
	Predicted Class	
Actual Class	1	0
1	201	85
0	25	2689

정확히 맞힌 것들

Classification Report 분류 리포트

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

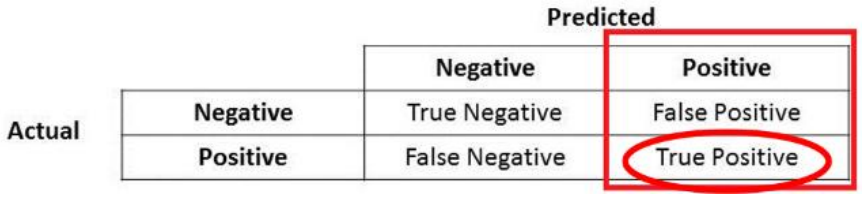
           0       1.00      1.00      1.00        11
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00         6

   micro avg       1.00      1.00      1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Precision and Recall 정밀도와 재현율

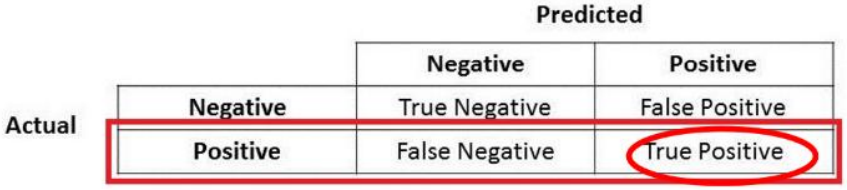
- 정밀도: 양성이라고 나온 사람 중에 진짜 양성인 사람 비율

$$\text{Precision} = \frac{True\ Positive}{True\ Positive + False\ Positive}$$



- 재현율: 진짜 양성인 사람 중에 양성이라고 나온 사람 비율

$$\text{Recall} = \frac{True\ Positive}{True\ Positive + False\ Negative}$$



- F1-Score: 정밀도와 재현율을 하나로 합친 점수

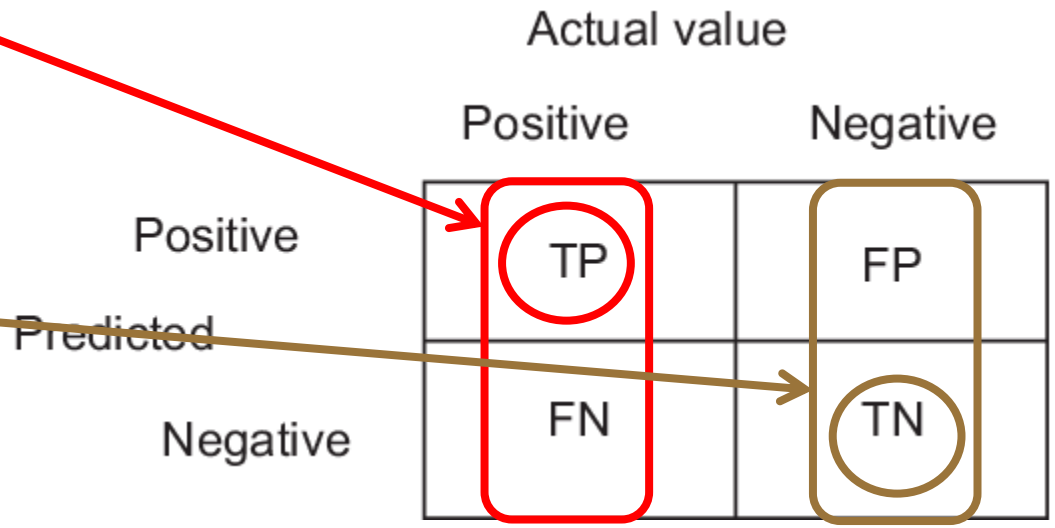
$$F1 = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

- F1-Score=0 -> Poor (P=0 or R=0)
- F1-Score=1 -> Perfect (P=1 or R=1)

Alternate Accuracy Measures
기타 정확도 측정치들

- Sensitivity 민감도
  - = true positive fraction
  - = TP / (TP + FN)
  - = 1 - false negative fraction

실제 양성이 진짜 양성에 속하는지

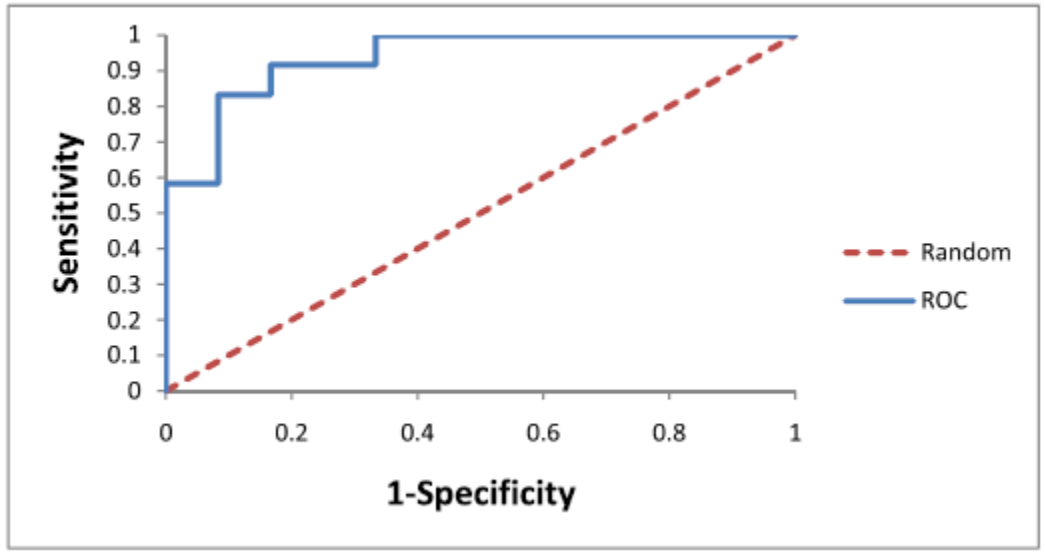


- Specificity 특이도
  - = true negative fraction
  - = TN / (TN + FP)
  - = 1 - false positive fraction

실제 음성이 진짜 음성에 속하는지

ROC Curve 락커브

True Positive Rate



False Positive Rate

False positive rate = 1- specificity

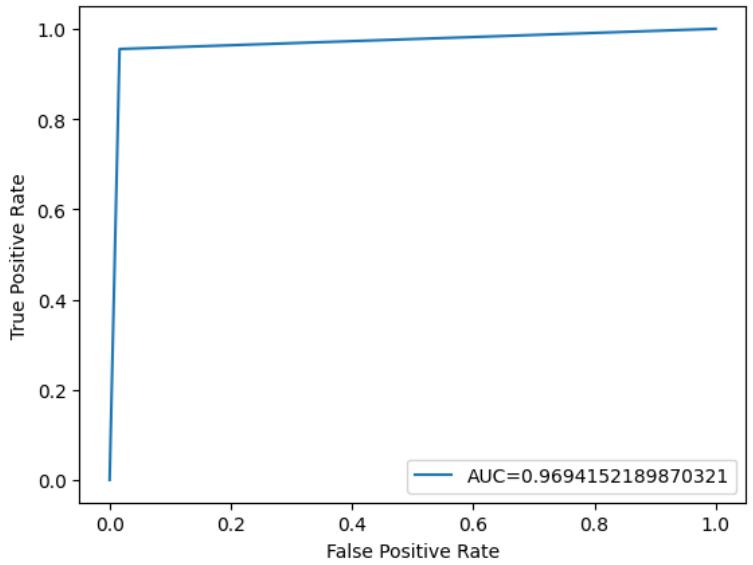
- Visualize the performance of a binary classification method.
  이진분류의 성능을 시각화

```
from sklearn.metrics import
plot_roc_curve
plot_roc_curve(cart, x_test, y_test)
```

ROC Curve 코드 락커브 코드

```
from sklearn.metrics import roc_curve,
roc_auc_score
fpr, tpr, _ = roc_curve(y_test,
grid_cv.predict(x_test))
auc = roc_auc_score(y_test,
grid_cv.predict(x_test))

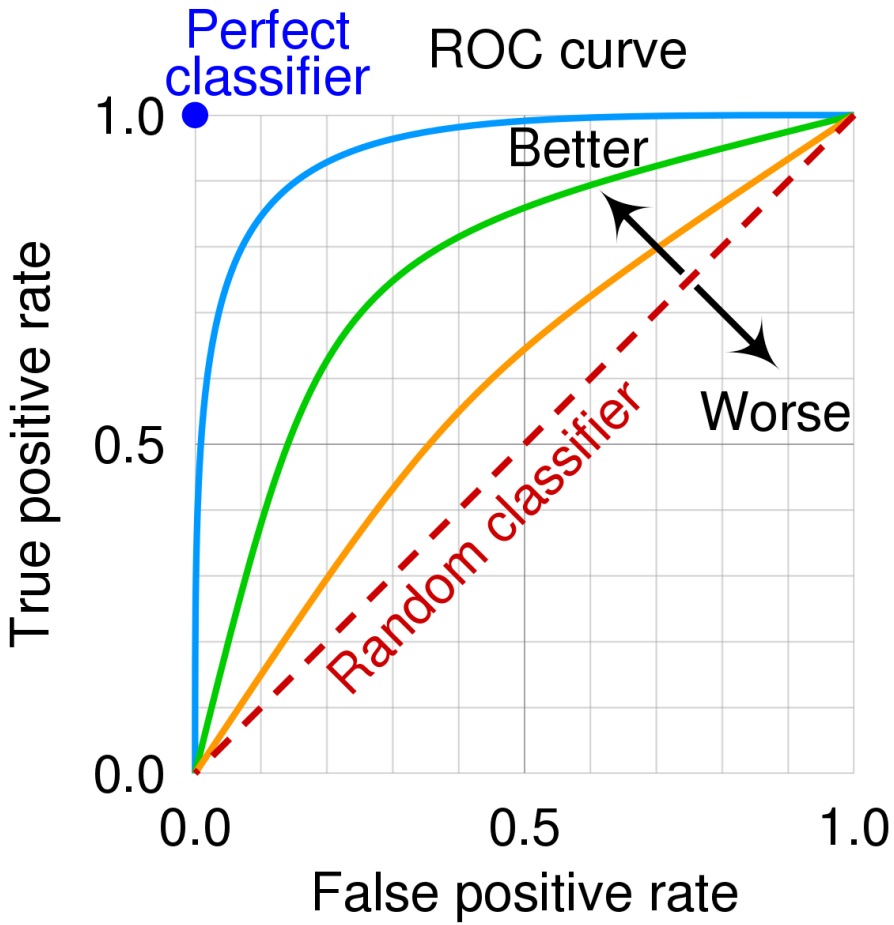
plt.plot(fpr,tpr,label="AUC="+str(auc))
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.legend(loc=4)
plt.show()
```



AUC (Area Under Curve)

- 곡선 밑의 면적으로 AUC
  면적이 클수록 모델의 정확도가
  높음

Area Under Curve	Evaluation
AUC >= .9	Excellent
.8 <= AUC < .9	Good
.7 <= AUC < .8	Fair
AUC < .7	Poor



Lab #5

- logit_train.csv 파일을 불러온 후 statsmodels를 이용해 로지스틱 회귀모델을 만들고, 합격 여부를 예측하시오.
  - gmat, gpa, work_experience를 입력 변수 X로 설정
  - admitted를 정답 변수 Y로 설정
  - 로지스틱 회귀 모델 학습
- logit_test.csv를 불러온 후 모델을 평가하시오.
  - 테스트 데이터의 합격 확률 예측
  - 예측값을 0 또는 1로 변환
  - 실제 합격 여부와 예측 결과 비교
  - 혼동행렬, 정확도 계산

Train Data 훈련 데이터

	gmat	gpa	work_experience	admitted
0	780	4	3	1
1	750	3.9	4	1
2	690	3.3	3	0
3	710	3.7	5	1
4	680	3.9	4	0
5	730	3.7	6	1
6	690	2.3	1	0
7	720	3.3	4	1
8	740	3.3	5	1
9	690	1.7	1	0
10	610	2.7	3	0
11	690	3.7	5	1
12	710	3.7	6	1
13	680	3.3	4	0
14	770	3.3	3	1
15	610	3	1	0
16	580	2.7	4	0
17	650	3.7	6	1
18	540	2.7	2	0
19	590	2.3	3	0
20	620	3.3	2	1
21	600	2	1	0
22	550	2.3	4	0
23	550	2.7	1	0
24	570	3	2	0
25	670	3.3	6	1
26	660	3.7	4	1
27	580	2.3	2	0
28	650	3.7	6	1
29	660	3.3	5	1
30	640	3	1	0
31	620	2.7	2	0
32	660	4	4	1
33	660	3.3	6	1
34	680	3.3	5	1
35	650	2.3	1	0
36	670	2.7	2	0
37	580	3.3	1	0
38	590	1.7	4	0
39	690	3.7	5	1

Test Data 테스트 데이터

	gmat	gpa	work_experience	admitted
0	590	2	3	0
1	740	3.7	4	1
2	680	3.3	6	1
3	610	2.3	1	0
4	710	3	5	1

Logistic Regression using Statsmodels
통계모델을 이용한 로지스틱 회귀

```
import statsmodels.api as sm
import pandas as pd
df = pd.read_csv('logit_train.csv', index_col = 0)
x_train = df[['gmat', 'gpa', 'work_experience']]
y_train = df[['admitted']] #x, y설정
log_reg = sm.Logit(y_train, x_train).fit() #모델구축
print(log_reg.summary()) #결과요약
```

Predictions 예측

```
# performing predictions on the test datdaset
y_pred = log_reg.predict(x_test)
prediction = list(map(round, y_pred)) #반올림


# comparing original and predicted values of y. 실제값, 예측값 출력
print('Acutal values', list(y_test.values))
print('Predictions :', prediction)
```

Evaluation 평가

```
from sklearn.metrics import confusion_matrix,
accuracy_score

# confusion matrix
cm = confusion_matrix(y_test, prediction)
print ("Confusion Matrix : \n", cm)

# accuracy score of the model
print('Test accuracy = ', accuracy_score(y_test,
prediction))
```

Lab #5

- logit_train.csv 파일을 불러온 후 sklearn LogisticRegression을 이용해 로지스틱 회귀모델을 만들고, 합격 여부를 예측하시오.
  - 로지스틱 회귀 모델 학습
  - 회귀 계수와 절편 확인
- logit_test.csv를 불러온 후 모델을 평가하시오.
  - 테스트 데이터 예측
  - 예측 정확도와 교차검증 점수 확인

Logistic Regression using sklearn
사이킷런을 이용한 로지스틱 회귀

- 이진문제에 유용. 다항문제도 가능

```
from sklearn.linear_model import
LogisticRegression
model =
LogisticRegression(multi_class='mul
tinomial', solver='lbfgs')
model.fit(x_train, y_train)
model.coef_
model.intercept_
```

```
##             Estimate Std. Error z value Pr(>|z|)
## Feature32   -0.8370     0.1760 -4.7558   0.0000
## Feature60    0.0379     0.0085  4.4684   0.0000
## Feature130   0.3404     0.0723  4.7072   0.0000
## Feature501   0.0010     0.0002  4.4735   0.0000
## Feature319   0.3263     0.0823  3.9641   0.0001
## Feature166  -0.5996     0.1662 -3.6083   0.0003
## Feature222 -12.6033     3.4697 -3.6324   0.0003
```

Predictions and Evaluation
예측 및 평가

```
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
from sklearn.model_selection import
cross_val_score
y_pred = model.predict(x_test) #테스트 데이터에 대한
예측값
model.score(x_train,y_train) #accuracy
accuracy_score(y_test, y_pred) #정확도
cross_val_score(model, x_test, y_test, cv=3,
scoring='accuracy') #교차검증
```

Classification Analysis Steps
분류분석 단계

Imports and configuration.
라이브러리 임포트 및 설정

- import, os.chdir()

Load the dataset.
파일 열기

- read_csv, read_excel, open (), requests.get()

Inspect the Dataset.
기본검사

- head(), dtypes, describe(), Isna().sum()

Visualize the Dataset.
데이터 탐색 및 시각화

- df.hist(), df.boxplot(), sns.heatmap(df.corr()), sns.pairplot() …

Classification Analysis Steps
분류분석 단계

Data cleaning and transformation.
데이터 클리닝 및 변환

- dropna(), fillna(), etc.

Splitting the Dataset. **X, Y**
데이터분리

- df[[]], df.drop(…, axis= 1)

Feature Scaling
스케일링

- MinMaxScaler, Standa rdScaler

Encoding **인코딩**

- Replace, LabelEncoder , OneHotEncoder, get_ dummies()

Classification Analysis Steps
분류분석 단계

Resample imbalanced data. **균형이 맞지 않는 데이터 리샘플링**

- RandomUnderSampler, RandomOverSampler, SMOTE

Splitting train and test data. **훈련 테스트데이터 분리**

- Train_test_split(x,y,test_size=.2, random_state=1)

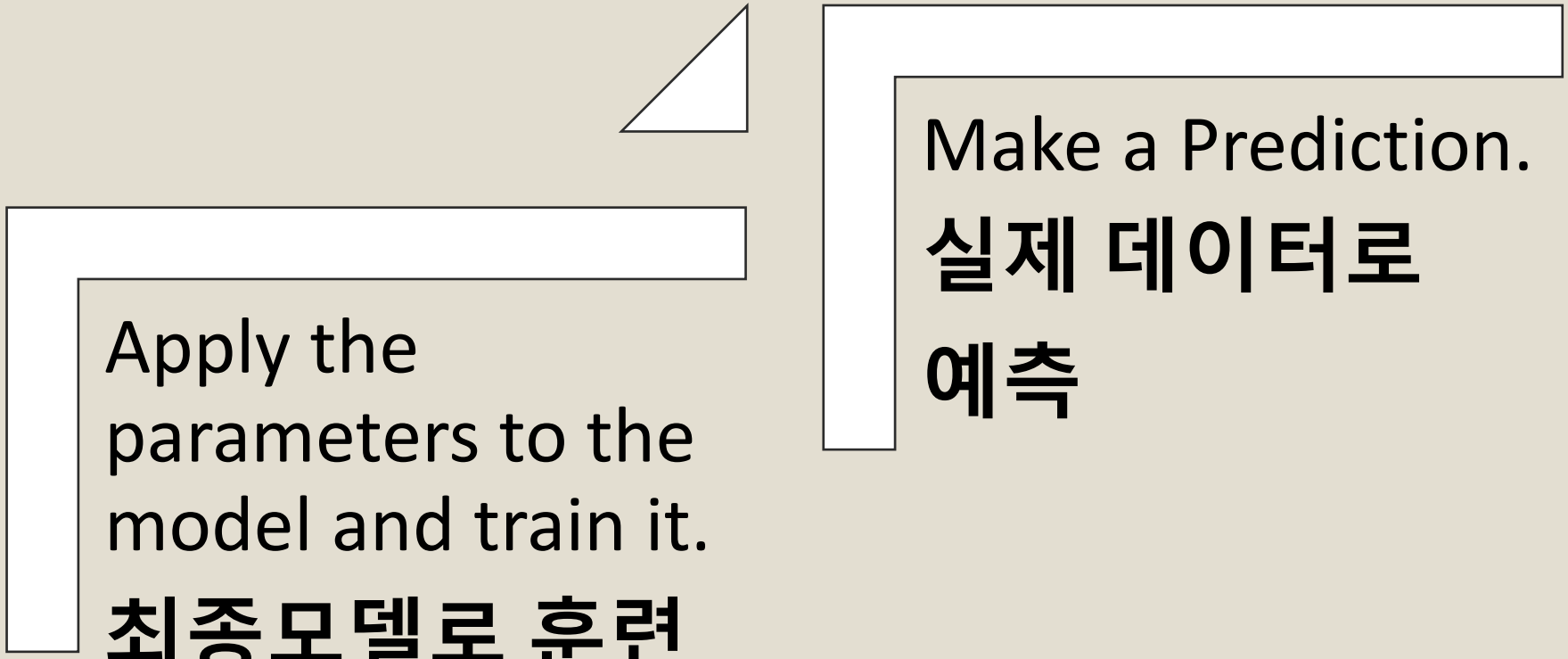
Comparing multiple algorithms. **알고리즘 적용 및 평가**

- LiniearRegression, LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, SVC, Kneighbors, LinearDiscrimantAnalysis, MLPClassifier
- accuracy_score, cross_val_score, confusion_matrix, classification_report, plot_roc_curve

Find the best parameters for SVC. **계수조정**

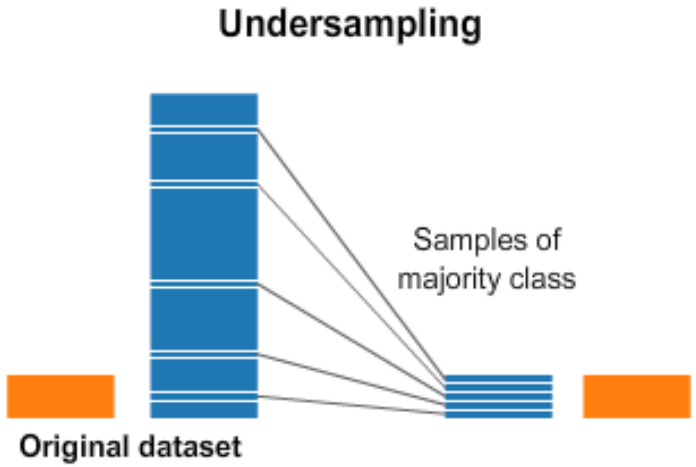
- For loop, GridSearchCV

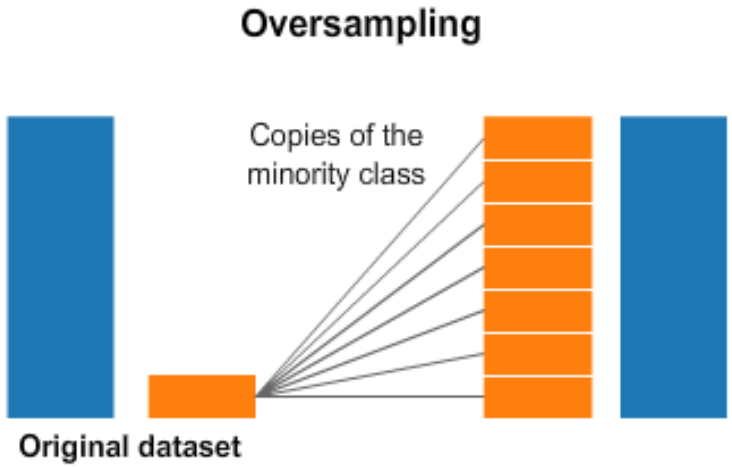
Classification Analysis Steps
분류분석 단계



Resampling 재샘플링

- A technique for dealing with highly unbalanced datasets. 불균형 데이터 세트를 처리하기 위한 기술
- Removing samples from the majority class (under-sampling) and / or adding more examples from the minority class (over-sampling). 다수의 클래스에서 샘플 제거 (언더샘플링) 하거나 소수의 클래스(오버샘플링)에서 더 많은 샘플 추가(오버샘플링)

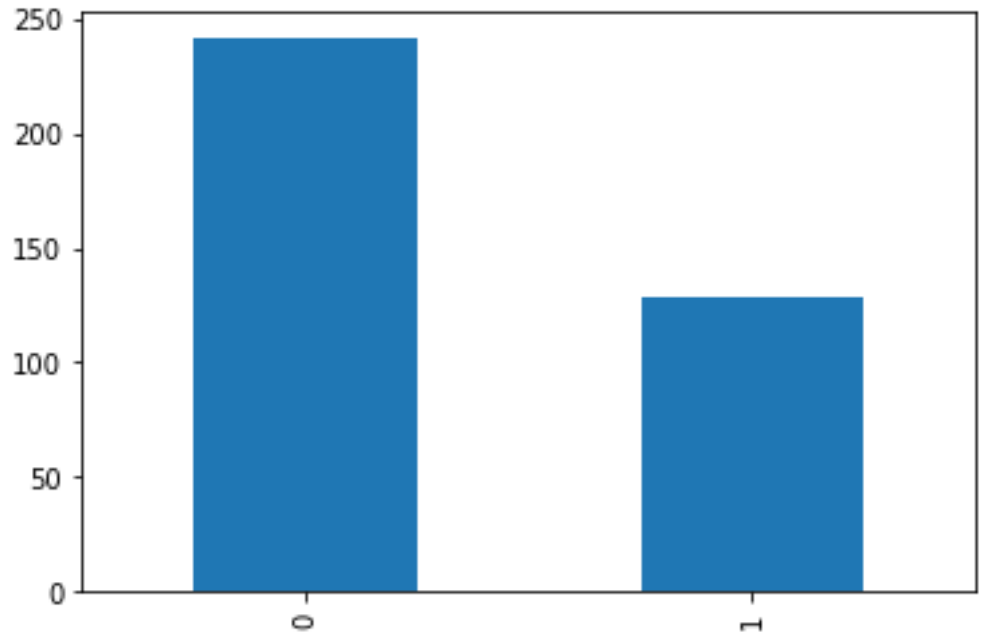




Python Imbalanced-Learn Module

- A python package offering a number of re-sampling techniques commonly used in datasets. 데이터 세트에 일반적으로 사용되는 여러 가지 재샘플링 기술을 제공하는 파이썬 패키지

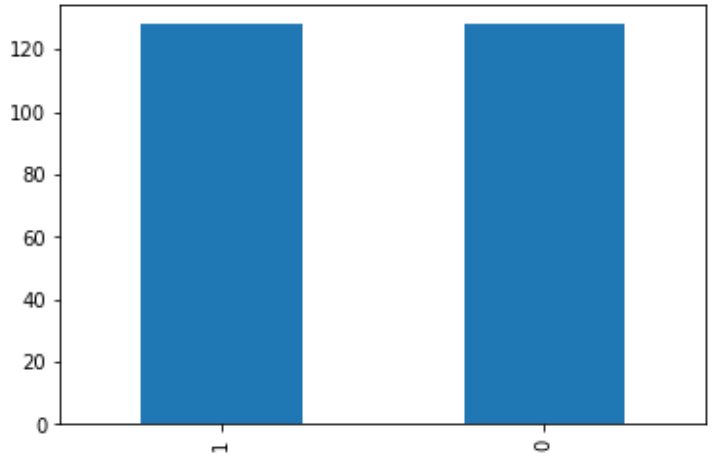
```
import imblearn
```



Random Under-Sampling
랜덤 언더샘플링

```
from imblearn.under_sampling import
RandomUnderSampler

rus =
RandomUnderSampler(return_indices=True)
X_rus, y_rus = rus.fit_sample(X, y)
```

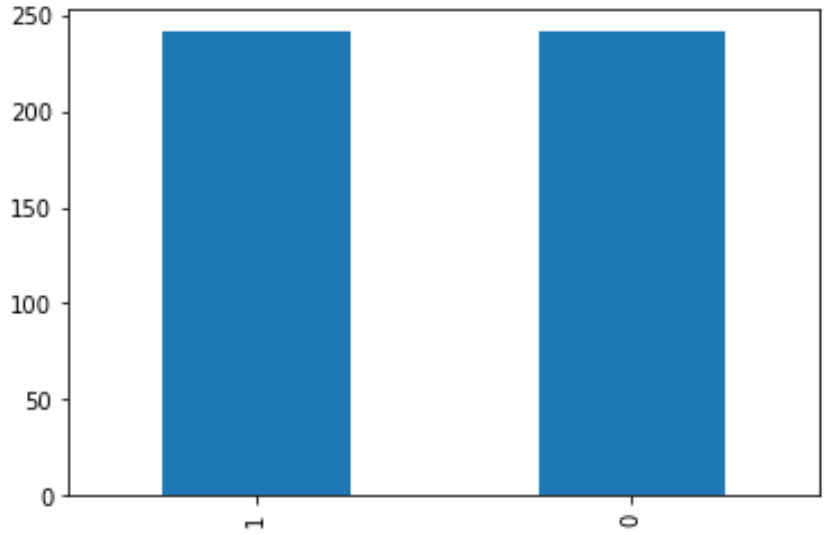


Random Over-Sampling
랜덤 오버샘플링

- 실패케이스를 복사하여 같은 갯수로 만듬

```
from imblearn.over_sampling import
RandomOverSampler

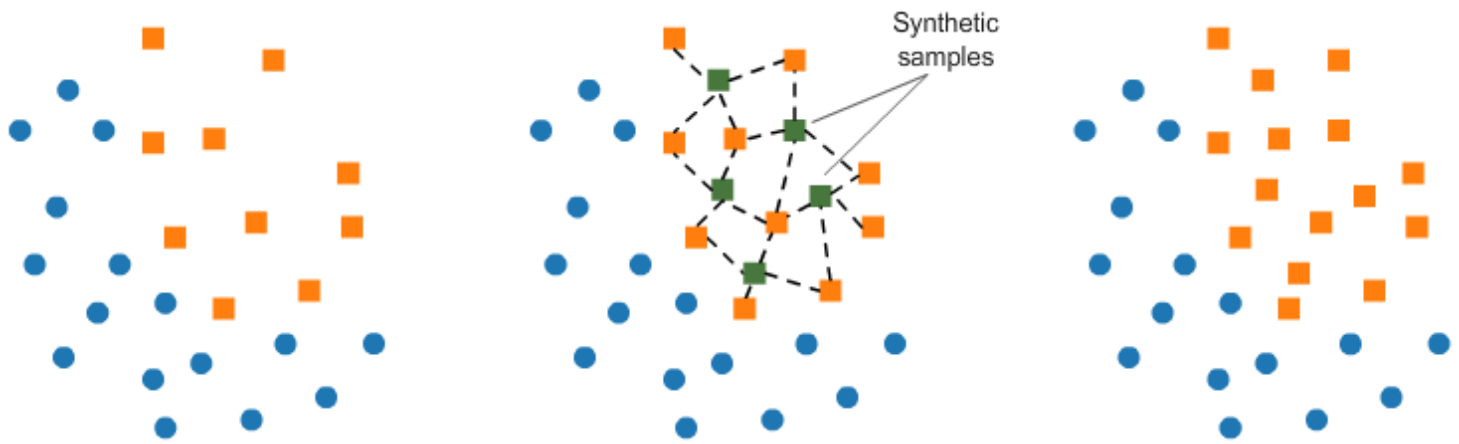
ros = RandomOverSampler()
X_ros, y_ros = ros.fit_sample(X, y)
```



SMOTE (Synthetic Minority Oversampling Technique)

- Works randomly pick a point from the minority class and computing the k-nearest neighbors for this point. The synthetic points are added between the chosen point and its neighbors. 소수클래스에서 무작위로 한 점을 선택하고 이 점에 대해 가장 가까운 이웃을 계산. 합성점은 선택한 점과 인접 지점 사이에 추가

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=27)
x_train, y_train = smote.fit_sample(x_train, y_train)
```



Hyperparameter Tuning
하이퍼파라미터 튜닝

- 모델의 성능을 높이기 위해 하이퍼파라미터 값을 조정하는 과정

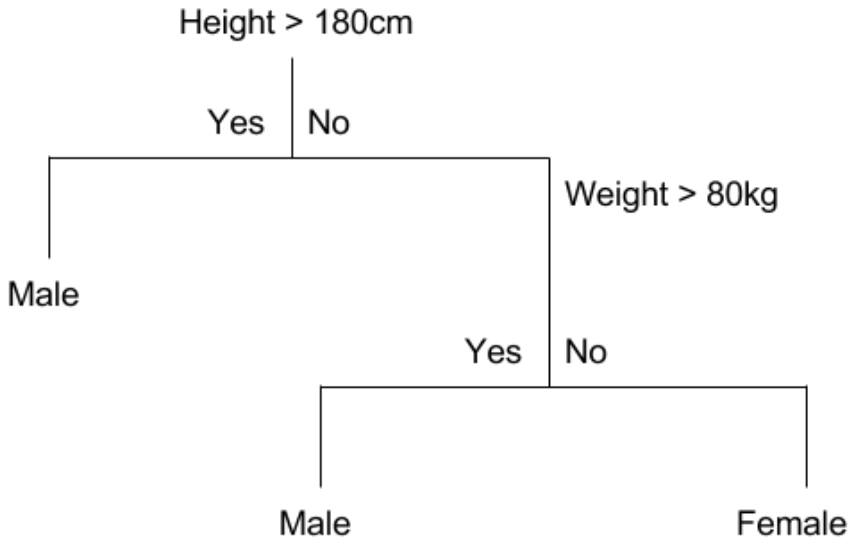
Algorithm	Hyperparameter
Decision Tree	max_depth, min_samples_splits, min_samples_leaf, criterion
Random Forest	n_estimators, min_samples_split, min_samples_leaf, max_features, max_depth , max_leaf_nodes
K-Nearest Neighbors	N_neighbors, weights, metric
Support Vector Machines	C, kernel, gamma, degree (for polynomial kernel)

Classification and Regression Trees (CART) 의사결정트리

- Find a tree that separates two or more classes of objects. 두개 이상의 클래스로 나누는 나무를 찾음

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
```

https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html



Split Criterion 분리기준

- The function to measure the quality of a split. 분리의 퀄리티를 측정하기 위한 함수
- Supported criteria are “gini” for the Gini impurity and “entropy” for the information gain. "지니"와 "엔트로피“를 사용할 수 있음

Gini Index 지니 인덱스

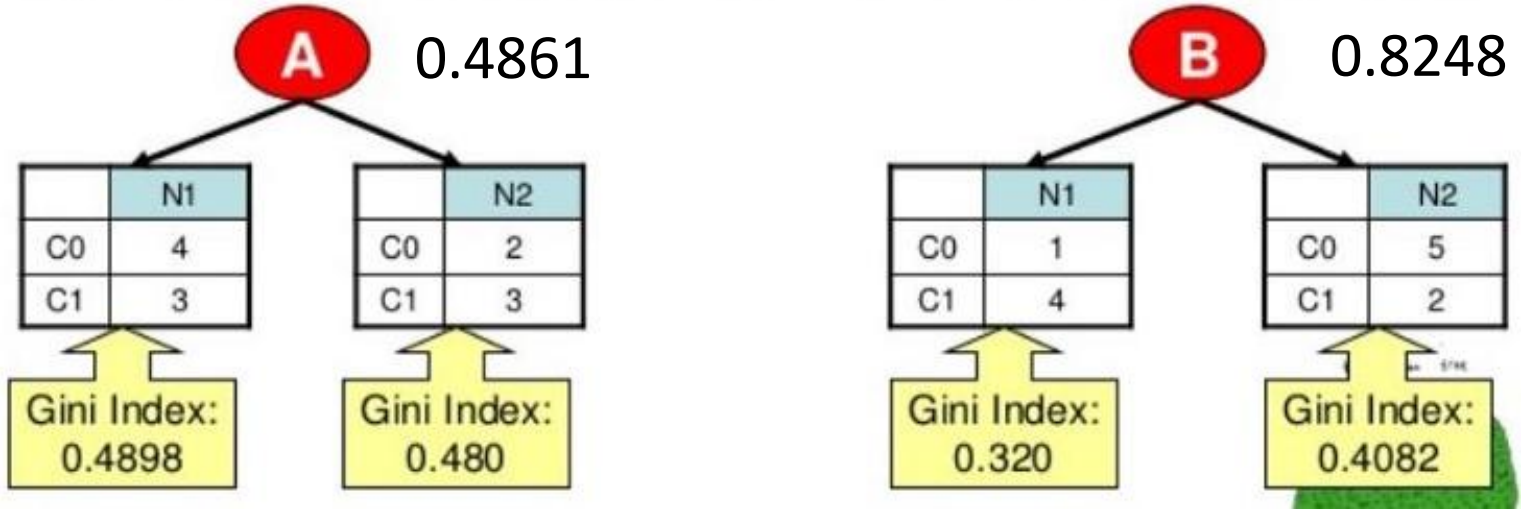
- Compute the weighted average of the Gini index of both attributes. 각 속성에 대한 지니 인덱스의 가중평균을 계산

$$= 1 - \sum_{t=0}^{t=1} P_t^2$$

Out of 14 instances ,
yes=9,no=5
1 - (9/14)^2- (5/14)^2

1-0.413-0.127= 0.46
Gini = 0.46

Suppose there are two ways (A and B) to split the data into smaller subset.



7/12 * 0.4898 + 5/12 * 0.480
= 0.4857

5/12 * 0.320 + 7/12 * 0.4082
= 0.3714

A Gini Impurity of 0 is the lowest and best possible impurity. 0값이 가장 불순도가 낮음. 잘 나눠졌다는 의미

Entropy 엔트로피

- A measure of homogeneity of the set of examples.
  집합의 동질성에 대한 측정치

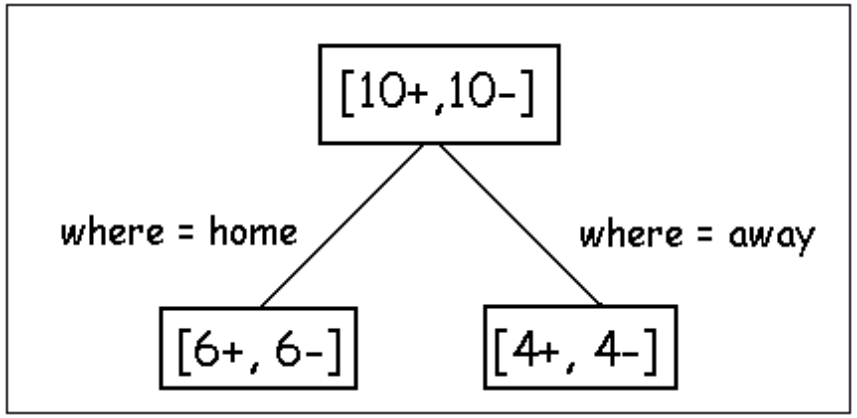
$$Entropy = \sum_{i=1}^{C} -p_i * \log_2(p_i)$$

- Suppose S has 25 examples, 15 positive and 10 negatives [15+, 10-]. Then the entropy of S relative to this classification is: 25개 중에 15개의 양수 및 10개의 음수가 있다고 가정함 [15+, 10-]. 그러면 이 분류에 대한 엔트로피는 다음과 같음
- E(S)=-(15/25) log2(15/25) - (10/25) log2(10/25) = 0.9709505944546686

Information Gain 정보이득

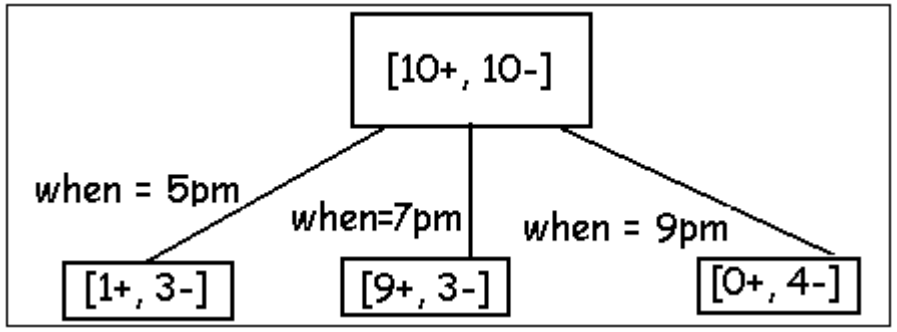
- Measures how well a given attribute separates the training examples according to their target classification. 주어진 속성이 훈련데이터를 클래스에 따라 잘 분류했는 지 측정
- This measure is used to select among the candidate attributes at each step while growing the tree. 이 측정치는 트리를 확장하는 동안 각 단계에서 후보 속성 중 하나를 선택하는 데 사용됨
- Measures the expected reduction in entropy, or uncertainty. 예상되는 엔트로피 감소 또는 불확실성을 측정

Entropy Calculation 엔트로피 계산



- Before partitioning, the entropy is
  - H(10/20, 10/20) = - 10/20 log(10/20) - 10/20 log(10/20) = 1
- Using the ``where" attribute, divide into 2 subsets. Entropy of the first set
  - H(home) = - 6/12 log(6/12) - 6/12 log(6/12) = .85
- Entropy of the second set
  - H(away) = - 4/8 log(6/8) - 4/8 log(4/8) = .71
- Expected entropy after partitioning
  - 12/20 * H(home) + 8/20 * H(away) = .80
- Information gain = 1 - 0.8 = 0.2

Entropy Calculation 엔트로피 계산

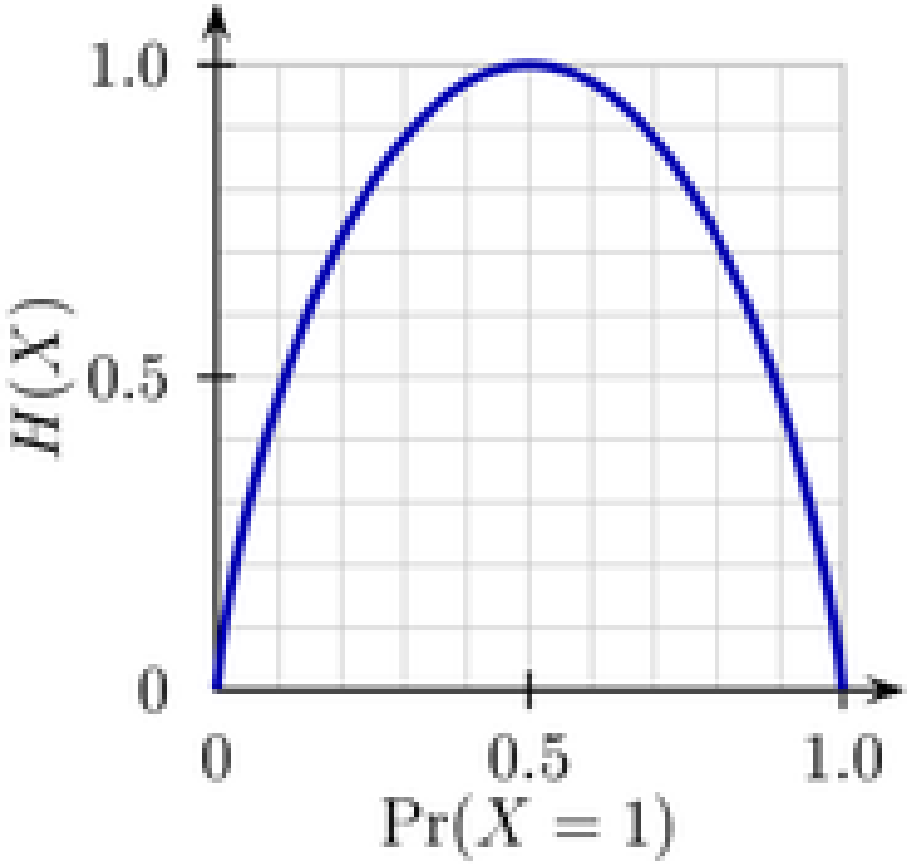


이전 단계와 비교했을 때 gain이 많은 쪽을 택함

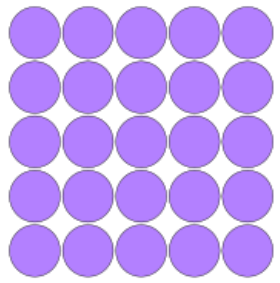
- Using the ``when" attribute, divide into 3 subsets. Entropy of the first set
  - H(5pm) = - 1/4 log(1/4) - 3/4 log(3/4) = .72
- Entropy of the second set
  - H(7pm) = - 9/12 log(9/12) - 3/12 log(3/12) = .81
- Entropy of the second set
  - H(9pm) = - 0/4 log(0/4) - 4/4 log(4/4) = 0
- Expected entropy after partitioning
  - 4/20 * H(1/4, 3/4) + 12/20 * H(9/12, 3/12) + 4/20 * H(0/4, 4/4) = 0.63
- Information gain = 1 - 0.63 = 0.37

Low vs High Entropy
낮은 대 높은 엔트로피

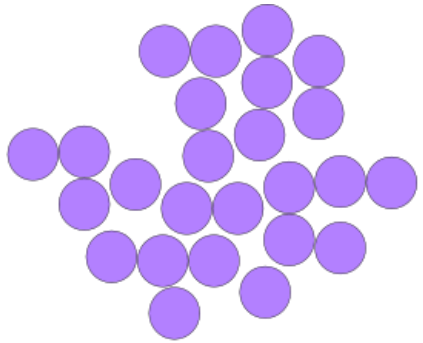
Entropy of a 2-class problem with regard to the portion of one of the two groups. 두 그룹 중 하나의 위치에 관한 클래스문제의 엔트로피



- The entropy is 0 if the outcome is ``certain". 아웃컴이 확실하면 엔트로피는 0
- The entropy is maximum if we have no knowledge of the system (or any outcome is equally possible). 만약 시스템에 대한 지식이 없거나 아웃컴이 똑같이 가능하면 엔트로피는 최대임



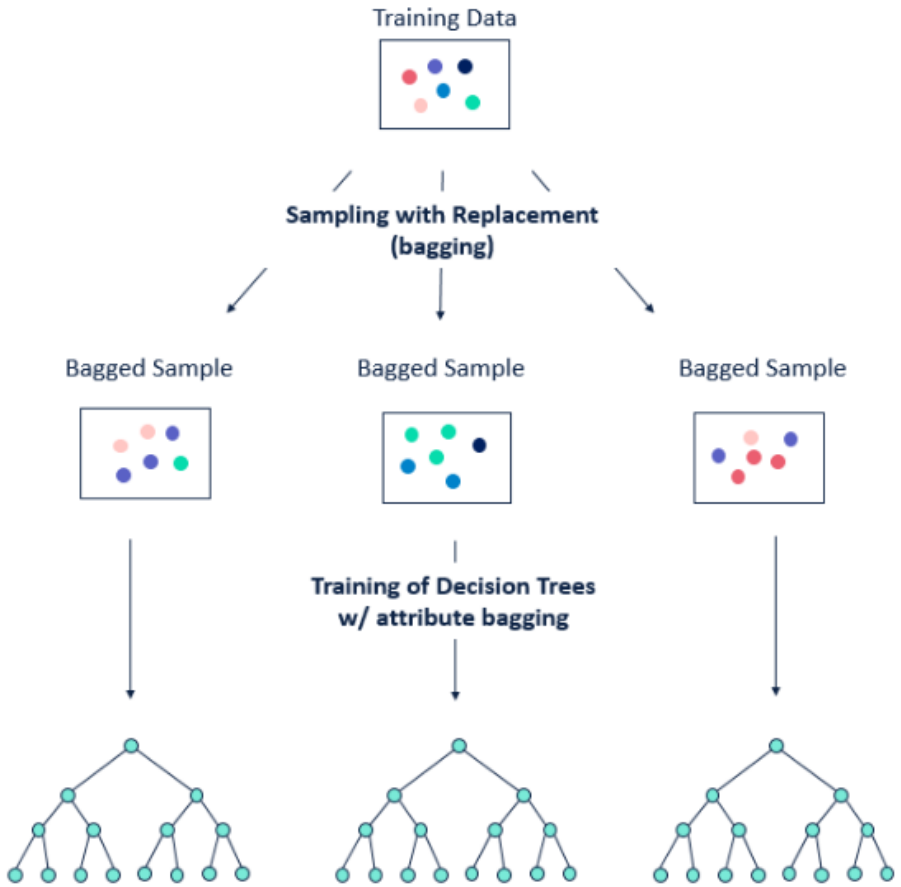
Low Entropy



High Entropy

RandomForest 랜덤포레스트

- 앙상블 알고리즘 중 비교적 수행속도가 빠름
- 하이퍼 파라미터 튜닝이 시간이 많이 소요됨




```
from sklearn.ensemble import
RandomForestClassifier
rf =
RandomForestClassifier(random_state=
0)
rf.fit(x_train, y_train)
pred = rf.predict(x_test)
accuracy = accuracy_score(y_test,
pred)
print('랜덤 포레스트 정확도:
{:.4f}'.format(accuracy))
```

sklearn.ensemble.RandomForestClassifier — scikit-learn 1.3.0 documentation

RandomForest Hyperparameter
랜덤포레스트 하이퍼 파라미터

- n_estimators - 결정트리의 갯수, default=10, 갯수를 늘리면 성능은 좋아지나 속도는 느려짐
- min_samples_split - 노드를 분할하기 위한 최소한의 샘플 데이터수, default=2, 작게 설정할 수 록 분할 노드가 많아져 과적합 가능성 증가
- min_samples_leaf - 리프노드가 되기 위해 필요한 최소한의 샘플 데이터수, 불균형 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있으므로 작게 설정
- max_features - 최적의 분할을 위해 고려할 최대 feature 개수, default='auto’
- max_depth - 트리의 최대 깊이, default = None, 깊이가 깊어지면 과적합
- max_leaf_nodes - 리프노드의 최대 개수

RandomForest Hyperparameter Tuning 랜덤포레스트 하이퍼 파라미터 튜닝

```
from sklearn.model_selection import
GridSearchCV

params = { 'n_estimators' : [10, 100],
           'max_depth' : [6, 8, 10, 12],
           'min_samples_leaf' : [8, 12, 18],
           'min_samples_split' : [8, 16, 20]
            }

# GridSearchCV 수행
rf = RandomForestClassifier(random_state =
0, n_jobs = -1)
grid_cv = GridSearchCV(rf, param_grid =
params, cv = 3, n_jobs = -1)
grid_cv.fit(x_train, y_train)

print('최적 하이퍼 파라미터: ',
grid_cv.best_params_)
print('최고 예측 정확도:
{:.4f}'.format(grid_cv.best_score_))
```

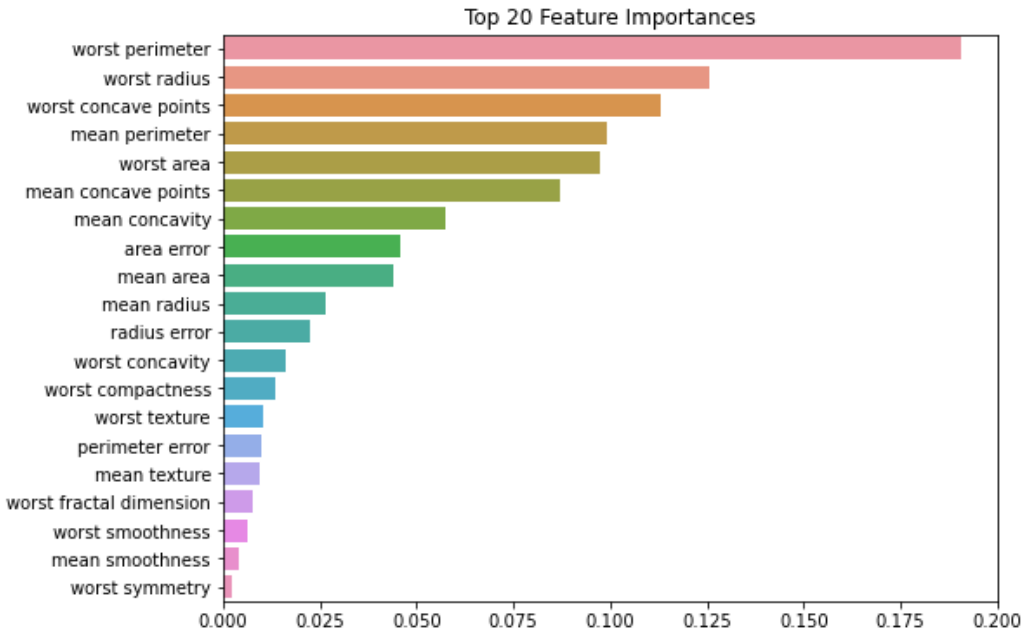
```
# 최적 하이퍼 파라미터로 다시 모델을 학습
rf2 = RandomForestClassifier(n_estimators =
100,
max_depth = 12,
min_samples_leaf = 8,
min_samples_split = 8,
random_state = 0,
n_jobs = -1)

rf2.fit(x_train, y_train)
pred = rf2.predict(x_test)
print('예측 정확도:
{:.4f}'.format(accuracy_score(y_test,pred))
)
```

Feature Importance Visualization
피처 중요도 시각화

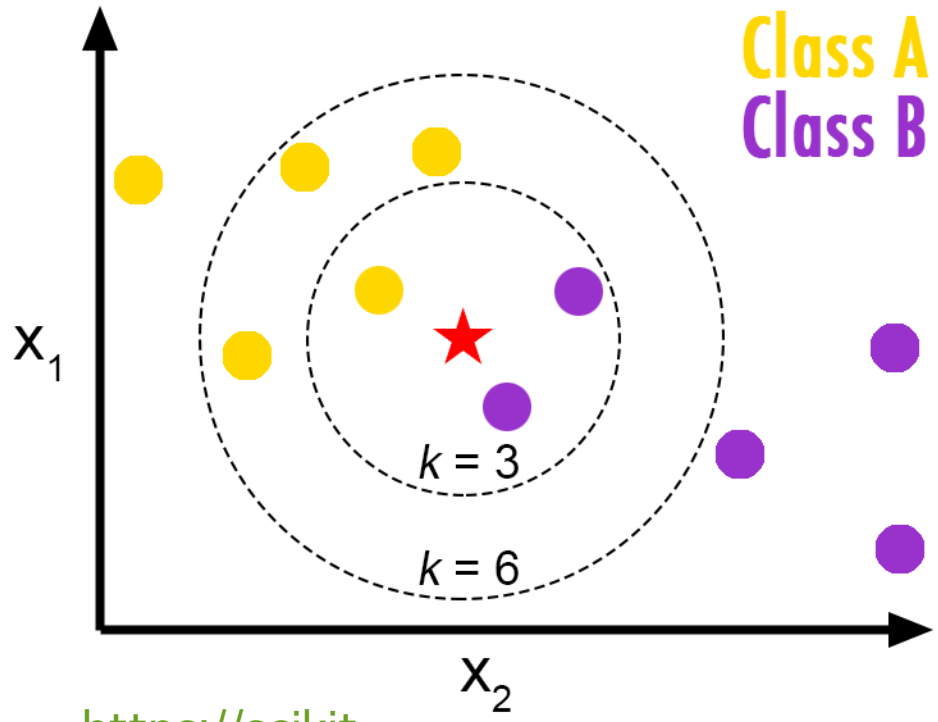
```
ftr_importances_values =
rf2.feature_importances_
ftr_importances =
pd.Series(ftr_importances_values, index =
x_train.columns)
ftr_top20 =
ftr_importances.sort_values(ascending=False
)[:20]

plt.figure(figsize=(8,6))
plt.title('Top 20 Feature Importances')
sns.barplot(x=ftr_top20, y=ftr_top20.index)
plt.show()
```



k-Nearest Neighbors (kNN)

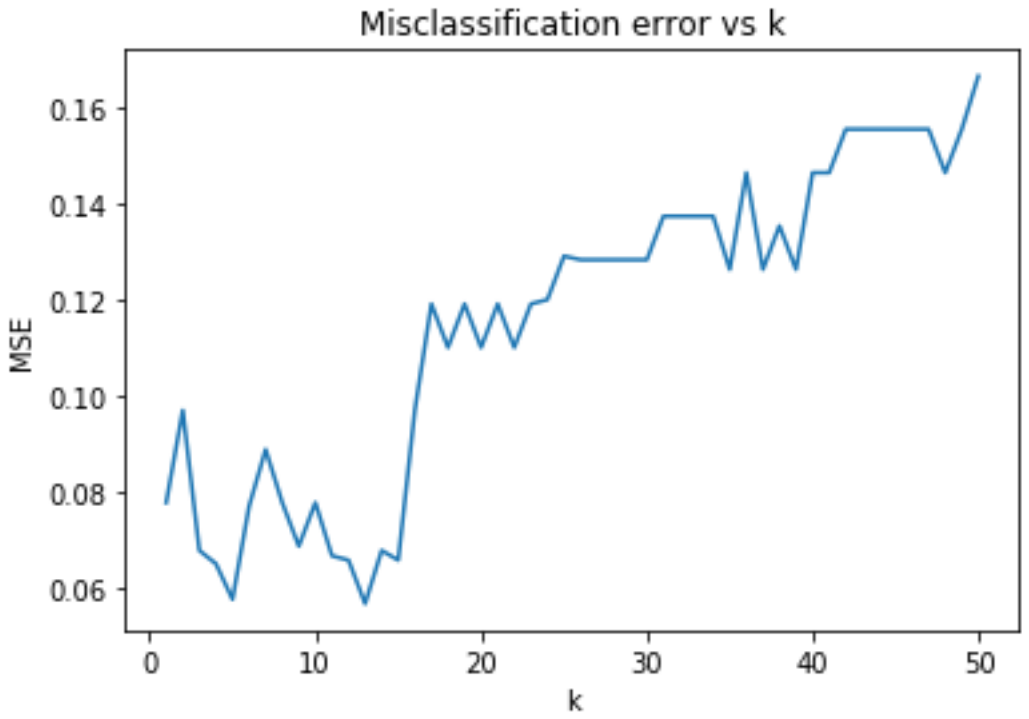
- The case is assigned to the class that most common amongst its K nearest neighbors measured by a distance function. 거리에 따라 가까운 k개의 이웃들을 정하고 그 이웃들의 클래스를 따름




```
from sklearn.neighbors import KNeighbo
orsClassifier
knn = KNeighborsClassifier()
knn.fit(x_train, y_train)
knn.predict(x_test)
```

https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

Deciding Number of Neighbors in kNN 적정 이웃의 갯수

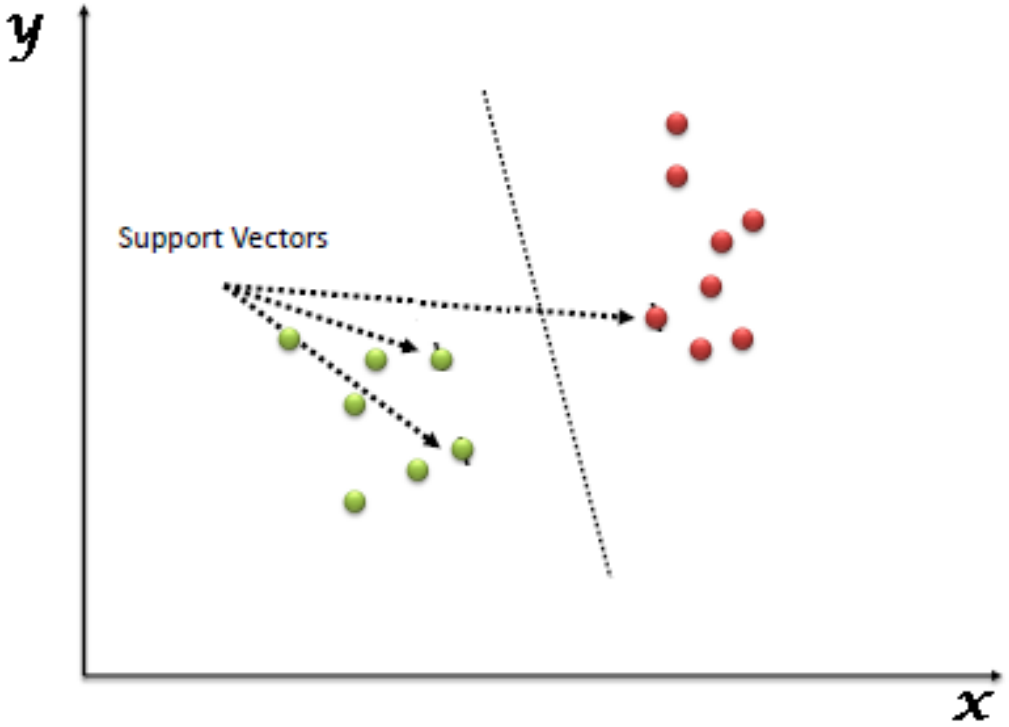



```
error.index(min(error))+1
```

```
neighbors = list(range(1, 50, 2))
cv_scores = []
for k in neighbors:
    knn =
KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train,
y_train, cv=10, scoring='accuracy')
    cv_scores.append(scores.mean())
mse = [1 - x for x in cv_scores]
optimal_k = neighbors[mse.index(min(mse))]
print("The optimal number of neighbors is
{}".format(optimal_k))
plt.plot(neighbors, mse)
plt.xlabel("Number of Neighbors K")
plt.ylabel("Misclassification Error")
plt.show()
```

Support Vector Machines (SVM)

- 데이터를 가장 잘 구분하는 경계선을 찾음
- 가장 가까운 데이터 포인트들의 거리를 최대화시킬 수 있는 하이퍼 플레인을 찾아 분류

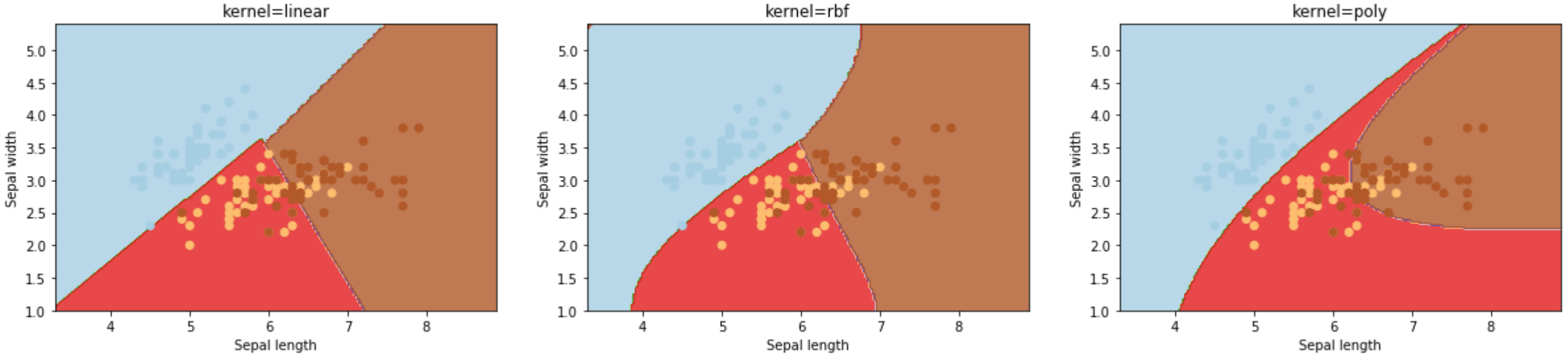



```
from sklearn.svm import SVC
svm = SVC()
svm.fit(x_train, y_train)
svm.predict(x_test)
```

https://scikit-learn.org/stable/api/sklearn.svm.html

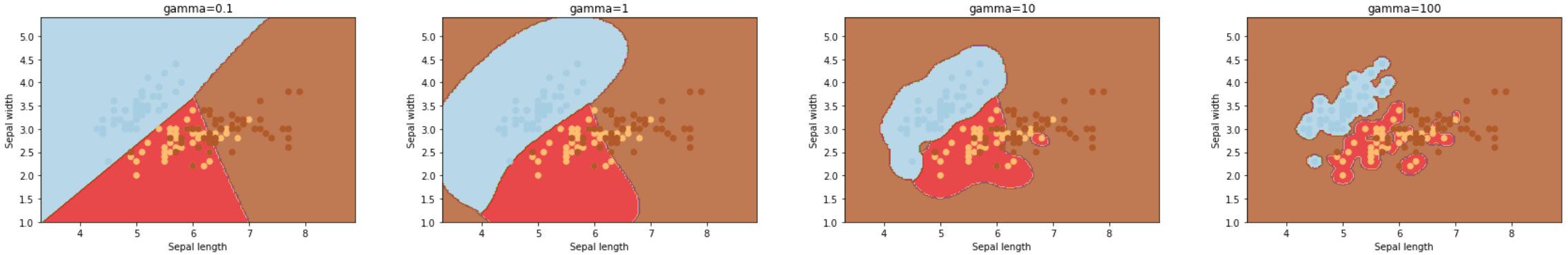
SVM Kernel 커널

- Kernel parameters selects the type of hyperplane used to separate the data. Using 'linear' will use a linear hyperplane (a line in the case of 2D data). 'rbf' and 'poly' uses a non linear hyper-plane. 커널은 데이터를 가를 때 사용하는 하이퍼플레인의 종류, rbf와 poly는 비선형 하이퍼플레인을 사용



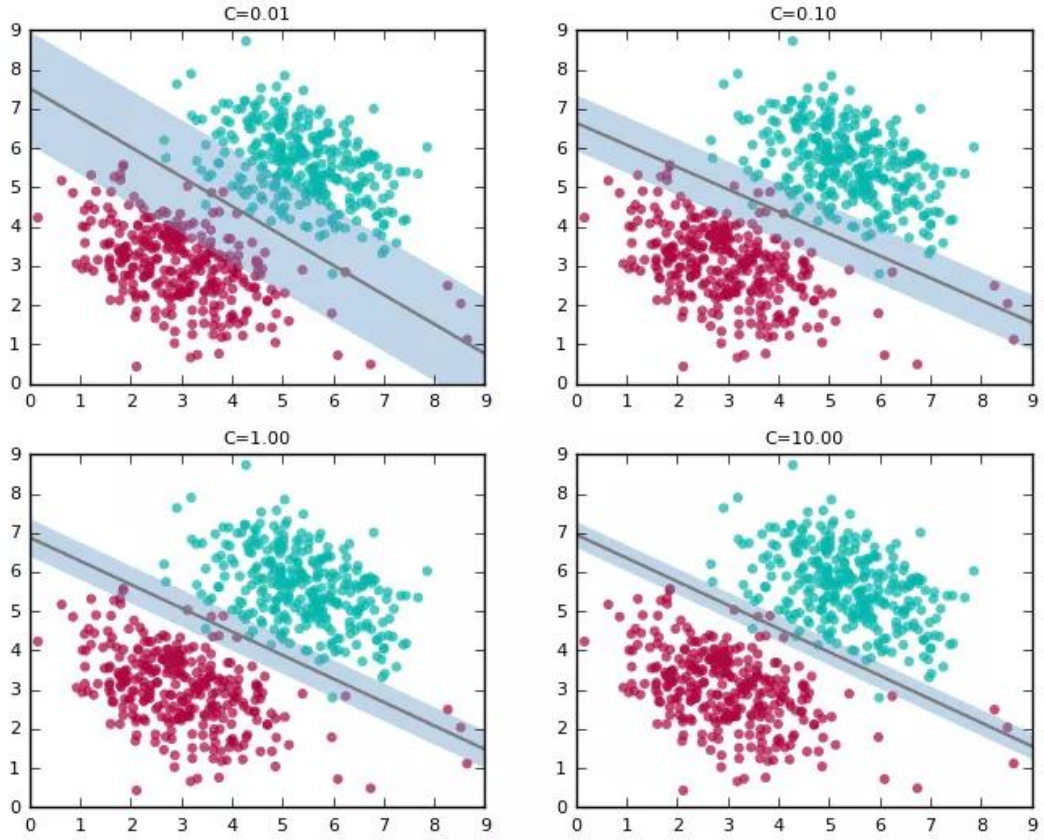
SVM Gamma 감마

- A parameter for non linear hyperplanes. The higher the gamma value it tries to exactly fit the training data set. 비선형 하이퍼플레인에 대한 계수. 감마가 높을수록 훈련데이터에 더 잘 맞음



SVM C

- A regularization parameter that can be adjusted to loosen or tighten the fit. 모델의 적합성을 느슨하게 또는 팽팽하게 조정할 수 있는 정규화 계수
- Lower values of C produce a wider margin with more errors on either side of the decision boundary. 낮은 C값은 마진이 넓어져서 오차가 더 많음
- Higher values yield a tighter fit to the training data with a correspondingly thinner margin and fewer errors. 높은 C값은 훈련데이터에 더 적합하고 마진이 좁아져서 오차가 더 적음



SVM Hyperparameter Tuning SVM 하이퍼 파라미터 튜닝

```
from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


# 1. 하이퍼파라미터 후보 설정

param_grid = {

    'C': [0.1, 1, 10, 100, 1000],

    'gamma': [1, 0.1, 0.01, 0.001, 0.0001],

    'kernel': ['rbf']

}


# 2. SVM 모델 생성

model = SVC()
```

```
# 3. 하이퍼파라미터 튜닝

grid = GridSearchCV(

    estimator=model,

    param_grid=param_grid,

    cv=5,

    scoring='accuracy'

)


# 4. 모델 학습 및 최적 파라미터 탐색

grid.fit(x_train, y_train)


# 5. 최적 파라미터 및 교차검증 정확도 출력

print("Best Parameters:", grid.best_params_)

print("Best Cross-validation Accuracy:", grid.best_score_)


# 6. 최적 모델로 테스트 데이터 예측

best_model = grid.best_estimator_

y_pred = best_model.predict(x_test)
```